

Thiết kế dữ liệu / cấu trúc smart contract

Nguồn: `contracts/RentalManager.sol` (ng nghiệp vụ) và `contracts/RentalAgreementToken.sol` (token ERC-721 đại diện một hợp đồng thuê — xem lý do tách 2 contract và lý do chọn ERC-721 tại [docs/lua-chon-token.md](#)).

1. Enum `Status` — vòng đời một tài sản/hợp đồng

Giá trị	Tên	Ý nghĩa
0	<code>Listed</code>	Chủ nhà đã đăng, chưa có người thuê.
1	<code>Active</code>	Người thuê đã đặt cọc, hợp đồng đang hoạt động.
2	<code>HandedOver</code>	Người thuê đã xác nhận bàn giao (trả phòng); có thể đang chờ chủ nhà đề xuất tất toán, hoặc chờ người thuê phản hồi đề xuất.
3	<code>Ended</code>	Đã tất toán cọc, hợp đồng kết thúc.
4	<code>Disputed</code>	Người thuê không đồng ý mức khấu trừ chủ nhà đề xuất — đang chờ trọng tài (<code>ARBITER_ROLE</code>) biểu quyết.
5	<code>Cancelled</code>	Chủ nhà đã hủy tin đăng (<code>cancelListing</code>) trong lúc còn <code>Listed</code> — vd đăng nhầm giá. Không thể thuê được nữa.

Chuyển trạng thái gần như một chiều: `Listed` → `Active` → `HandedOver` → { `Ended` | `Disputed` → `Ended` }, hoặc `Listed` → `Cancelled` — không có đường lùi thật sự (từ `Disputed` chỉ đi tiếp tới `Ended` , không quay lại `HandedOver` ; `Cancelled` là trạng thái cuối, không đi tiếp đâu nữa). Mỗi hàm public chỉ cho phép gọi khi contract đang ở đúng trạng thái liền trước (kiểm tra bằng `require`).

2. Struct `Property`

Trường	Kiểu	Mô tả
<code>landlord</code>	<code>address</code>	Chủ nhà, gán khi <code>listProperty</code> , không đổi.
<code>title</code>	<code>string</code>	Mô tả tài sản (vd "Phòng trọ Quận 1, 25m²").
<code>location</code>	<code>string</code>	Vị trí/khu vực.
<code>monthlyRent</code>	<code>uint256</code>	Tiền thuê mỗi kỳ, đơn vị wei.
<code>deposit</code>	<code>uint256</code>	Tiền cọc yêu cầu, đơn vị wei.
<code>status</code>	<code>Status</code>	Trạng thái hiện tại.
<code>tenant</code>	<code>address</code>	Người thuê hiện tại, <code>address(0)</code> khi chưa có ai thuê.
<code>depositHeld</code>	<code>uint256</code>	Số tiền cọc đang thực sự nằm trong contract (giảm về 0 sau <code>endLease</code>).
<code>startedAt</code>	<code>uint256</code>	Thời điểm <code>rentProperty</code> thành công (block timestamp).

Trường	Kiểu	Mô tả
<code>rentPaidCount</code>	<code>uint256</code>	Số kỳ đã trả tiền thuê thành công.
<code>imageCID</code>	<code>string</code>	Một hoặc nhiều CID/URL ảnh phòng lưu trên IPFS , cách nhau bằng dấu phẩy nếu nhiều ảnh; rỗng (<code>" "</code>) nếu chủ nhà không đính kèm ảnh. Vẫn chỉ là 1 field <code>string</code> — không đổi kiểu dữ liệu on-chain khi hỗ trợ nhiều ảnh. Contract chỉ lưu chuỗi tham chiếu — file ảnh thật nằm trên IPFS, không lưu on-chain.
<code>note</code>	<code>string</code>	Ghi chú tự do của chủ nhà (nội quy, lưu ý...), có thể rỗng (<code>" "</code>).
<code>nextDueDate</code>	<code>uint256</code>	Hạn trả tiền thuê kỳ tiếp theo (unix seconds). Gán khi <code>rentProperty</code> (<code>startedAt + rentPeriod</code>), cộng thêm <code>rentPeriod</code> mỗi lần <code>payRent</code> thành công. Qua hạn mà chưa trả → <code>payRent</code> bắt buộc cộng thêm phạt trễ.
<code>proposedDeduction</code>	<code>uint256</code>	Mức khấu trừ tiền cọc chủ nhà đề xuất khi tắt toán (wei), gán bởi <code>proposeSettlement</code> .
<code>settlementProposed</code>	<code>bool</code>	Đã có đề xuất tắt toán đang chờ người thuê phản hồi (<code>acceptSettlement</code> / <code>disputeSettlement</code>) hay chưa.

`propertyCount` (`uint256`) và `mapping(uint256 => Property) properties` lưu toàn bộ tài sản, id bắt đầu từ 1.

Cấu hình đặt lúc deploy (`immutable` , không đổi được sau khi lên chain)

Biến	Kiểu	Ý nghĩa	Giá trị mặc định (Ignition)
<code>rentPeriod</code>	<code>uint256</code>	Chu kỳ tính hạn trả tiền thuê (giây).	2.592.000 (30 ngày)
<code>lateFeeBps</code>	<code>uint256</code>	Mức phạt trả trễ, phần vạn (basis points).	500 (5%)
<code>arbiterApprovalsRequired</code>	<code>uint256</code>	Số trọng tài cần đồng thuận cùng 1 mức khấu trừ mới thi hành khi tranh chấp.	2

Vai trò (`AccessControl`)

Vai trò	Ai được cấp
<code>DEFAULT_ADMIN_ROLE</code>	Địa chỉ <code>admin</code> truyền vào constructor (có thể <code>grantRole(ARBITER_ROLE, ...)</code> cho thêm trọng tài).
<code>ARBITER_ROLE</code>	<code>admin</code> (mặc định lúc deploy) + bất kỳ địa chỉ nào được <code>admin</code> cấp thêm.

3. Event (nguồn cho tab "Lịch sử")

Event	Khi nào emit	Tham số
<code>PropertyListed</code>	<code>listProperty</code> thành công	<code>id</code> , <code>landlord</code> , <code>title</code> , <code>monthlyRent</code> , <code>deposit</code>
<code>Rented</code>	<code>rentProperty</code> thành công	<code>id</code> , <code>tenant</code> , <code>depositPaid</code> , <code>startedAt</code>

Event	Khi nào emit	Tham số
RentPaid	payRent thành công	id, tenant, amount, latePenalty, paidAt
HandoverConfirmed	confirmHandover thành công	id, tenant, confirmedAt
SettlementProposed	proposeSettlement thành công	id, deductAmount
DisputeRaised	disputeSettlement thành công	id, tenant
DisputeVoteCast	voteOnDispute thành công	id, arbiter, deductAmount, voteCount
LeaseEnded	acceptSettlement hoặc voteOnDispute (khi đủ phiếu) thành công	id, refundToTenant, deductToLandlord, endedAt
ListingCancelled	cancellisting thành công	id, landlord

Frontend (`ChainRentalService.loadHistory()`) gọi `contract.queryFilter()` cho tất cả event này rồi gộp + sắp xếp theo `blockNumber` giảm dần — đây là cách "theo dõi lịch sử thanh toán" mà không cần một database off-chain riêng.

4. Hàm public và điều kiện (`require`)

Hàm	Ai gọi được	Điều kiện chặn	Hiệu ứng tiền
<code>listProperty(title, location, monthlyRent, deposit, imageCID, note)</code>	Bất kỳ ai (trở thành chủ nhà)	<code>monthlyRent > 0</code>	Không chuyển tiền. <code>imageCID</code> / <code>note</code> có thể truyền "" nếu không dùng.
<code>cancellisting(id)</code>	Đúng <code>landlord</code>	tài sản tồn tại; đang <code>Listed</code> (chưa ai đặt cọc)	Không chuyển tiền — chuyển sang <code>Cancelled</code> , tin không hiển thị thuê được nữa.
<code>rentProperty(id) payable</code>	Bất kỳ ai trừ chủ nhà	tài sản tồn tại; đang <code>Listed</code> ; <code>msg.sender != landlord</code> ; <code>msg.value == deposit</code>	ETH gửi kèm ở lại trong contract (<code>depositHeld</code>); đồng thời mint 1 <code>RentalAgreementToken</code> với <code>tokenId = id</code> cho người thuê.
<code>payRent(id) payable</code>	Đúng <code>tenant</code>	đang <code>Active</code> ; <code>msg.sender == tenant</code> ; <code>msg.value == monthlyRent</code> (+ phạt trễ nếu quá <code>nextDueDate</code>)	ETH chuyển thẳng cho <code>landlord</code> qua <code>call{value:}</code> .
<code>confirmHandover(id)</code>	Đúng <code>tenant</code>	đang <code>Active</code> ; <code>msg.sender == tenant</code>	Không chuyển tiền.

Hàm	Ai gọi được	Điều kiện chặn	Hiệu ứng tiền
<code>proposeSettlement(id, deductAmount)</code>	Đúng <code>landlord</code>	đang <code>HandedOver</code> ; <code>deductAmount <= depositHeld</code>	Không chuyển tiền — chỉ ghi nhận đề xuất.
<code>acceptSettlement(id)</code>	Đúng <code>tenant</code>	đang <code>HandedOver</code> ; đã có đề xuất	Tất toán theo đúng mức đề xuất (xem <code>_settle</code>).
<code>disputeSettlement(id)</code>	Đúng <code>tenant</code>	đang <code>HandedOver</code> ; đã có đề xuất	Không chuyển tiền — chuyển sang <code>Disputed</code> .
<code>voteOnDispute(id, deductAmount)</code>	Địa chỉ có <code>ARBITER_ROLE</code>	đang <code>Disputed</code> ; <code>deductAmount <= depositHeld</code> ; chưa vote lần nào cho tranh chấp này	Nếu đủ <code>arbiterApprovalsRequired</code> phiếu cùng 1 mức → tất toán theo mức đó (xem <code>_settle</code>).
<code>getProperty(id) / getAllProperties()</code>	Bất kỳ ai (view)	—	—

`_settle(id, deductAmount)` (nội bộ, không phải hàm public): chuyển `deductAmount` cho `landlord`, phần còn lại (`depositHeld - deductAmount`) hoàn cho `tenant`, chuyển trạng thái sang `Ended`. Được gọi bởi cả `acceptSettlement` lẫn `voteOnDispute` — nghĩa là có **đúng 2 con đường** dẫn tới `Ended`: người thuê tự đồng ý, hoặc đủ trọng tài đồng thuận.

5. Vì sao tiền cọc giữ ở contract còn tiền thuê chuyển thẳng?

- **Tiền cọc** cần một bên trung gian khách quan giữ cho đến khi có xác nhận bàn giao — đây chính là điểm khác biệt so với thuê nhà truyền thống (chủ nhà giữ cọc, người thuê phải tin tưởng). Contract đóng vai trò "ký quỹ" (escrow).
- **Tiền thuê định kỳ** không có tranh chấp về việc "ai giữ" — trả xong là xong nghĩa vụ kỳ đó, nên chuyển thẳng cho chủ nhà ngay khi `payRent` thành công, giảm số dư đọng trong contract (giảm bề mặt tấn công/rủi ro nếu contract có lỗi).

6. Bảo vệ reentrancy

Tất cả hàm có chuyển ETH (`rentProperty`, `payRent`, `acceptSettlement`, `voteOnDispute`) đều có modifier `nonReentrant` (`@openzeppelin/contracts/utils/ReentrancyGuard.sol`) và tuân theo pattern **checks-effects-interactions**: cập nhật state (`p.status`, `p.depositHeld`, `p.rentPaidCount`, `p.nextDueDate`) **trước** khi gọi `.call{value: ...}()` chuyển ETH — nhờ vậy dù bên nhận là một contract độc hại cố gọi lại (`re-enter`) cũng không khai thác được state cũ. Việc mint token trong `rentProperty` cũng đặt **sau** khi cập nhật state + emit event, theo đúng nguyên tắc này (xem mục 7 bên dưới).

7. Contract `RentalAgreementToken` (token đại diện hợp đồng thuê)

ERC-721 (`OpenZeppelin ERC721 + AccessControl`), tên `"Rental Agreement"`, ký hiệu `"LEASE"`. Không kế thừa từ `RentalManager`, được `RentalManager` gọi sang qua địa chỉ lưu tại biến `immutable agreementToken`.

Vai trò

Vai trò	bytes32	Ai được cấp
DEFAULT_ADMIN_ROLE	0x00	Địa chỉ deploy (admin truyền vào constructor)
MINTER_ROLE	keccak256("MINTER_ROLE")	Địa chỉ RentalManager (cấp qua Ignition module sau khi deploy)

Hàm

Hàm	Ai gọi được	Ghi chú
mintAgreement(tenant, propertyId)	Chỉ MINTER_ROLE	Mint token tokenId = propertyId cho tenant . Revert AccessControlUnauthorizedAccount nếu gọi sai vai trò.
transferFrom / safeTransferFrom (kế thừa từ ERC721)	—	Luôn revert TransferNotAllowed() — token khoá vĩnh viễn sau khi mint (override _update).
approve / setApprovalForAll	—	Luôn revert TransferNotAllowed() — không cần approve vì không thể chuyển nhượng.
ownerOf(tokenId) (kế thừa)	Bất kỳ ai (view)	Trả về địa chỉ người thuê đã/đang giữ hợp đồng thuê đó — không đổi kể cả sau endLease .

Không có hàm burn — token tồn tại vĩnh viễn kể cả sau khi hợp đồng Ended , đóng vai trò bằng chứng lịch sử "đã từng thuê tài sản này", giống nguyên tắc "không xoá lịch sử" ở ví dụ chứng chỉ.

Sơ đồ quan hệ

